

Contact Form → Email Functionality

A build walkthrough for slmcustomart.com | Prepared July 9, 2026 | Web3Forms edition

1. What's There Right Now

The inquiry form lives on **pages/contact.html** (class order - form). It collects first name, last name, email, phone, portrait type, and a message, and has a Submit button. Right now it's markup only:

- No name or id attributes on the inputs, so no script can currently read the values.
- No action, method, or JS submit handler — clicking Submit does nothing.
- The site is fully static (HTML/CSS/vanilla JS, no framework, no package.json, no existing server).
- The site is hosted on Cloudflare, but nothing about this approach depends on that — it would work on any static host.

Since you'd rather keep this simple, the plan below skips writing any server code at all.

2. Recommended Approach: Web3Forms

Between the two no-backend options, **Web3Forms** is the better fit for “as simple as possible”:

	Web3Forms	EmailJS
Setup	Enter your email, get an access key. Done.	Connect an email account (OAuth), then build an email template in their editor.
Free tier	250 submissions / month	200 requests / month
Submission record	Stored 30 days in their dashboard — a free backup log	Not stored by default
Spam protection	Built-in honeypot field, optional hCaptcha	None built in
Front-end code	One hidden field + a form post	SDK script + service ID + template ID + public key

How a submission flows: visitor fills out the form → front-end JS posts the data straight to Web3Forms' API → Web3Forms emails it to slmcustomart@gmail.com and logs a copy in your Web3Forms dashboard → visitor sees a “Thanks, message sent” confirmation. No server code, no hosting changes, no environment variables.

One trade-off worth knowing: submissions aren't saved to your own Supabase project, only kept in Web3Forms' dashboard for 30 days (free plan). If you later want a permanent record inside your own database, or want inquiries to show up in the admin page, that's a small addition on top of this (a Cloudflare Function that also writes to Supabase) — but it isn't necessary to get emails flowing, and can be added later without redoing any of this work.

3. Step-by-Step Build

STEP 1 Create a Web3Forms access key

- Go to web3forms.com and click “Create your Form.”
- Enter the destination inbox — slmcustomart@gmail.com (the address already listed on the contact page) — and verify it via the confirmation email Web3Forms sends.
- Copy the Access Key it gives you. This key is safe to use in public front-end code — it only works as a mailing alias, it can't read your inbox or send from your account.

STEP 2 Update the HTML form

Add the hidden access key field, name attributes on every input so the data has readable labels in the email, a honeypot field for spam, an id for the JS hook, and a status line for feedback. Updated form markup for `contact.html`:

```
<form class="order-form" id="contact-form">

  <!-- Web3Forms access key (safe to be public) -->
  <input type="hidden" name="access_key" value="YOUR_ACCESS_KEY_HERE">
  <input type="hidden" name="subject" value="New Portrait Inquiry – SLM Custom Art">
  <input type="hidden" name="from_name" value="SLM Custom Art Website">

  <div class="form-row">
    <div class="form-group">
      <label>First Name *</label>
      <input type="text" name="First Name" id="firstName" required>
    </div>
    <div class="form-group">
      <label>Last Name *</label>
      <input type="text" name="Last Name" id="lastName" required>
    </div>
  </div>
  <div class="form-group">
    <label>Email *</label>
    <input type="email" name="email" id="email" required>
  </div>
  <div class="form-group">
    <label>Phone</label>
    <input type="tel" name="Phone" id="phone">
  </div>
  <div class="form-group">
    <label>Portrait Type *</label>
    <select name="Portrait Type" id="portraitType" required>
      <option value="">Select an option</option>
      <option>Home Portrait</option>
      <option>Pet Portrait</option>
    </select>
  </div>
  <div class="form-group">
    <label>Tell me about your vision – I'll bring it to life *</label>
    <textarea name="message" id="message" rows="5" required></textarea>
  </div>

  <!-- Honeypot: real visitors never see or fill this in -->
```

```



```

Note the fields use human-readable name values like “First Name” and “Portrait Type” — Web3Forms uses these directly as the labels in the notification email, so there's no template to design.

STEP 3 Wire up the front-end script

Add a new file `pages/contact.js` so the form submits in the background (no page reload) and shows an inline confirmation, matching the rest of the site's feel.

```

// pages/contact.js
document.addEventListener("DOMContentLoaded", () => {
  const form = document.getElementById("contact-form");
  const statusEl = document.getElementById("form-status");
  if (!form) return;

  form.addEventListener("submit", async (e) => {
    e.preventDefault();

    const submitBtn = form.querySelector(".order-submit");
    submitBtn.disabled = true;
    statusEl.textContent = "Sending...";

    const payload = Object.fromEntries(new FormData(form).entries());

    try {
      const res = await fetch("https://api.web3forms.com/submit", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          "Accept": "application/json"
        },
        body: JSON.stringify(payload)
      });
      const result = await res.json();

      if (result.success) {
        statusEl.textContent = "Thanks! Your message has been sent.";
        form.reset();
      } else {
        statusEl.textContent = result.message || "Something went wrong. Please try again.";
      }
    } catch (err) {
      statusEl.textContent = "Network error. Please try again later.";
    } finally {
      submitBtn.disabled = false;
    }
  });
});

```

Add before the closing `</body>` tag in `contact.html`, next to the existing `index.js` include:

```
<script src="contact.js"></script>
```

STEP 4 Test it

- Open `contact.html` (locally or on a preview deploy) and submit the form with real-looking test data.
- Confirm the email arrives at `slmcustomart@gmail.com` — check the spam folder the first time.
- Log into `app.web3forms.com` to see the submission recorded in the dashboard.
- Try submitting with a required field left blank to confirm the browser's built-in validation still blocks it.

STEP 5 Deploy

Commit the `contact.html` changes and the new `contact.js`, then push to GitHub as usual. There's nothing to configure on the Cloudflare side — no environment variables, no functions folder, no build changes. The form works the moment the files are live.

4. Optional Hardening (nice-to-have, not required for MVP)

- **hCaptcha** — Web3Forms has a built-in free hCaptcha option if the honeypot field alone doesn't stop spam; it's a couple of extra lines in the form plus one script tag.
- **Custom redirect** — an optional hidden `redirect` field can send visitors to a dedicated “Thank you” page instead of (or in addition to) the inline message, if that's ever preferred.
- **Upgrade path** — if inquiries need to live permanently in your own database or show up in the admin page down the road, that's a later addition (a small Cloudflare Function that also writes to Supabase) and doesn't require reworking anything built here.

5. Quick Checklist

- Create a Web3Forms access key for `slmcustomart@gmail.com`
- Add the access key, name attributes, honeypot field, and status line to the `contact.html` form
- Create `pages/contact.js` with the fetch-based submit handler
- Link `contact.js` in `contact.html`
- Test a real submission and confirm the email + dashboard entry
- Push to GitHub — no other deploy steps needed
- (Optional) Add hCaptcha if spam becomes an issue

Prepared for Alex — `slmcustomart` project. This document is an implementation guide only; no changes were made to the site.